Appello 5 febbraio 2016



Politecnico di Milano

Anno accademico 2014-2015

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:

Esercizio 1

Si consideri la classe `Mazzo` di carte (da bridge). L'universo di tutte le possibili carte può contenere anche carte di altro tipo per esempio carte da UNO. Ogni mazzo contiene inizialmente 52 carte, 13 per ogni seme (cuori = 0, quadri = 1, fiori = 2 e picche = 3) e 4 per ogni valore (da 1 a 13). Si definisca in JML:

1. l'invariante privato della classe `Mazzo`, ipotizzando che esista una classe `Carta` e che il rep sia un `ArrayList<Carta> carteMazzo`. La classe `Carta` fornisce anche due metodi `int seme()` e `int valore()` che restituiscono il seme e il valore di ogni carta come interi (secondo la numerazione esposta sopra).

2. la specifica del metodo `pesca()` che restituisce una `Carta` a caso dal mazzo.

3. la specifica del metodo `mescola()` che cambia l'ordine delle carte.

Per rispondere ai punti 2 e 3, si supponga che `Mazzo` offra un metodo `carte()` che restituisce le carte rimaste nel mazzo come un `ArrayList<Carta>`.

Sempre usando JML, sarebbe poi possibile specificare che due, o più, invocazioni successive del metodo `pesca()` restituiscono carte diverse?

SOLUZIONE:

PUNTO 1;

```
//@private invariant
//@ carteMazzo!=null && carteMazzo.size()<=52
// nessuna carta e' null
//@ (\forall int i; i>=0&&i<carteMazzo.size(); carteMazzo.get(i)!=null)
// ogni carta contenuta deve essere di uno dei semi consentiti
//@ (\forall Carta c; carteMazzo.contains(c); c.seme()>=0 && c.seme()<4)
// ogni carta contenuta deve essere di uno dei valori consentiti
//@ (\forall Carta c; carteMazzo.contains(c); c.valore()>=0 && c.valore()<14)
// ogni carta appare al piu' una volta
//@ (\forall int i; i>=0 && i<carteMazzo.size(); \forall int j; j>i && j<carteMazzo.s
    (carteMazzo.get(i).seme() != carteMazzo.get(j).seme() ||
    carteMazzo.get(i).valore() != carteMazzo.get(j).valore()))
```

PUNTO 2

```
//@requires carte.size()> 1
//@ensures \result!=null &&
// la carta c'era prima
//@ (\old(carte().contains(\result))) &&
// la carta viene rimossa
//@ !(carte().contains(\result)
// le altre carte devono rimanere uguali
//@ (\forall Carta c; \old(carte().contains(c)) &&
//@ (c.valore()!=\result.valore() ||
//@ c.seme()!=\result.seme()); carte().contains(c)) &&
// non posso aggiungere carte duplicate (vincolo sulla dimensione)
//@ carte().size()==\old(carte().size()-1)
```

PUNTO 3

```
//@requires carte.size()> 2
//@ensures
// la dimensione non cambia (in realta' implicata dalle seguenti)
//@ \old(carte().size())==carte().size()
```

```
// tutte le carte di prima ci sono anche adesso
//@ (\forall Carta c; \old(carte().contains(c));
//@ (\exists Carta d; carte().contains(d);
//@ d.valore()==c.valore() && d.seme()==c.seme());
// tutte le carte di adesso c'erano anche prima
//@ (\forall Carta c; carte().contains(c);
//@ (\exists Carta d; \old(carte().contains(d));
//@ d.valore()==c.valore() && d.seme()==c.seme());
```

No, in JML è possibile specificare le pre e post condizioni riguardanti la singola esecuzione del metodo. La proprietà potrebbe essere tuttavia implicata (come nel caso corrente).

Esercizio 2

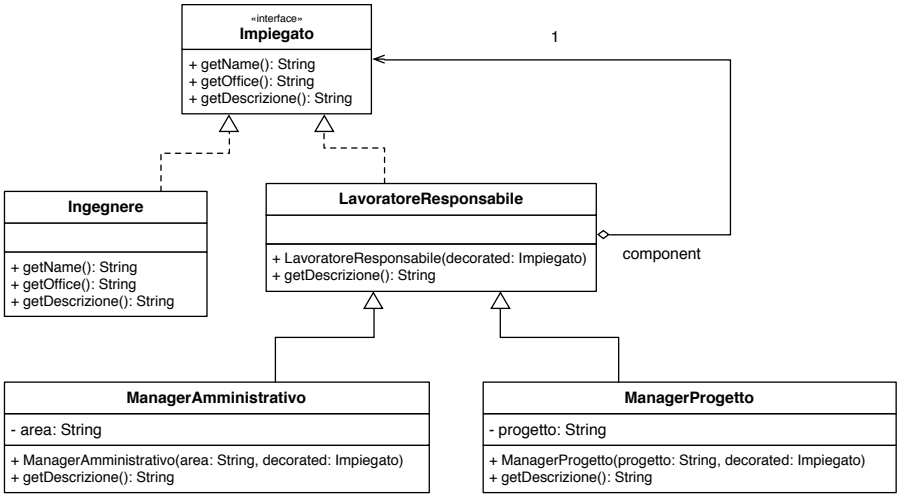
Si consideri l'insieme di impiegati di un'azienda. Gli impiegati espongono tre metodi `String getName()` e `String getOffice()` che ritornano nome e ufficio degli impiegati e `String getDescrizione()` che ritorna le mansioni dell'impiegato.

Ci sono vari tipi di impiegato, per esempio gli ingegneri. Le responsabilità degli impiegati (si considerino per semplicità solo gli ingegneri) possono cambiare dinamicamente. In particolare, un ingegnere può avere la responsabilità di manager amministrativo o manager di progetto. Il comportamento del metodo `String getDescrizione()` viene modificato opportunamente. Per esempio, se un ingegnere `ing` è manager amministrativo di un'area A la stringa "Manager di A" viene concatenata alla descrizione ritornata dall'invocazione del metodo `getDescrizione()`. Se a `ing` viene anche aggiunta la funzionalità di manager amministrativo dell'area B, ottenendo l'oggetto `ing1`, il metodo `getDescrizione()` invocato su `ing1` concatena la stringa "Manager di B" alla descrizione di `ing`. Infine se a `ing1` viene aggiunta la funzionalità manager del progetto $P1$, ottenendo l'oggetto `ing2`, la stringa ottenuta invocando il metodo `getDescrizione()` sull'oggetto `ing2` sarà "Oltre ad essere $\langle$ XXX $\rangle$ sono Manager di $P1$", dove XXX è la stringa ritornata dal metodo `getDescrizione()` di `ing1`. Come evidenziato dagli esempi, un ingegnere può essere manager amministrativo di più aree e/o manager di più progetti.

1. Si modelli, attraverso un diagramma delle classi UML e un design pattern opportuno, una soluzione al problema sopra presentato.
2. Si scriva anche la struttura del codice Java risultante. Ovvero, si definiscano le classi identificate al passo precedente, le loro relazioni e le intestazioni dei metodi principali.
3. Si scriva il codice Java che crea un'istanza di un oggetto ingegnere con funzionalità di manager amministrativo per l'area A e per l'area B e project manager del progetto P1.

SOLUZIONE:

1. È possibile usare un pattern Decorator.



2.
```
public interface Impiegato {
String getName();
String getDescrizione();
String getOffice();
}

public class Ingegnere implements Impiegato {
@Override
public String getName(){ ... }

@Override
public String getDescrizione(){ ... }

@Override
public String getOffice(){ ... }

}

public class LavoratoreResponsabile implements Impiegato {

private Impiegato decorated;

public LavoratoreResponsabile(Impiegato decorated){ ... }

@Override
public String getName(){ ... }

@Override
public String getDescrizione(){ ... }

@Override
public String getOffice(){ ... }



}

public class ManagerAmministrativo extends LavoratoreResponsabile {
```

```
    private String area;

    public ManagerAmministrativo(String area, Impiegato decorated){ ... }

    @Override
    public String getDescrizione(){
            return "Manager di "+area+super.getDescrizione();
        }

    }

    public class ManagerProgetto extends LavoratoreResponsabile {
        private String progetto;

        public ManagerProgetto(String progetto, Impiegato decorated){ ... }

        @Override
        public String getDescrizione(){
                return "Oltre ad essere <"+super.getDescrizione()+">"
                            +" sono anche Manager di "+progetto;
        }

    }
```

3. ```
   Ingegnere base = new Ingegnere(...);
   Impiegato managerP = new ManagerProgetto("P1", base);
   Impiegato managerBP = new ManagerAmministrativo("B", managerP);
   Impiegato managerABP = new ManagerAmministrativo("A", managerBP);
   ```

Esercizio 3

Un gruppo di 50 studenti deve sostenere un esame orale all'università. Tre docenti sono a disposizione degli studenti, ma chiaramente possono interrogare uno studente per volta. Per semplicità, si supponga che ogni esame abbia una durata casuale. Alla fine dell'esame ogni docente propone un voto allo studente e, se questo lo accetta, il voto viene verbalizzato.

Si completi il seguente programma (concorrente) Java che simuli la situazione appena descritta. Il singleton `Aula` svolge il ruolo di coordinatore. Lo `Studente` cercherà di sostenere l'esame, ma dovrà mettersi in attesa se tutti i docenti fossero impegnati. Il `Docente` inizierà un esame e, dopo un tempo variabile, proporrà un voto. Lo `Studente` potrà accettarlo e quindi il voto verrà verbalizzato dal `Docente`.

Si noti che lo studente deve completare le intestazioni dei metodi, ove necessario, e il corpo dei metodi `iniziaEsame`, `terminaEsame` e `partecipaAEsame`.

```
import java.util.Stack;

public class Aula {
    public Stack<Docente> freeDocenti;

    public Aula(){
        this.freeDocenti=new Stack<Docente>();
        this.freeDocenti.push(new Docente());
        this.freeDocenti.push(new Docente());
        this.freeDocenti.push(new Docente());
    }
    public static void main(String[] args) {
```
```

```
        Aula a=new Aula();
        for(int i=0; i<50; i++){
            new Studente().partecipaAEsame(a);
        }
    }

    public             Docente iniziaEsame() throws InterruptedException{









    }

    public             void terminaEsame(Docente d){









    }

    public             void sostieniEsame(Studente s) throws InterruptedException{

        Docente d=iniziaEsame();
        int voto=d.valuta(s);
        if(voto>18 && s.accetta(voto)){
            d.verbalizza(s, voto);
        }
        terminaEsame(d);
    }
}

import java.util.Random;

public class Docente {

    public             int valuta(Studente s) throws InterruptedException{
        Thread.sleep(new Random().nextInt(300));
        return new Random().nextInt(30);
    }
    public             void verbalizza(Studente s, int voto){
    }
}

import java.util.Random;

public class Studente {
```

```
    public             boolean accetta(int voto){
        return new Random().nextBoolean();
    }

    public             void partecipaAEsame(Aula a){
        new Thread(new Runnable() {
```

```
    }
}
```

```
package prova;

import java.util.Stack;

public class Aula {
    public Stack<Docente> freeDocenti;

    public Aula(){
        this.freeDocenti=new Stack<Docente>();
        this.freeDocenti.push(new Docente());
        this.freeDocenti.push(new Docente());
        this.freeDocenti.push(new Docente());
    }
    public static void main(String[] args) {

        Aula a=new Aula();
        for(int i=0; i<50; i++){
            new Studente().partecipaAEsame(a);
       }
   }

   public synchronized Docente iniziaEsame() throws InterruptedException{
       while(freeDocenti.isEmpty()){
           wait();
       }
       return freeDocenti.pop();
   }
   public synchronized void terminaEsame(Docente d){
```

```
        freeDocenti.push(d);
        this.notifyAll();
    }
    public void sostieniEsame(Studente s) throws InterruptedException{

        Docente d=iniziaEsame();
        int voto=d.valuta(s);
        if(voto>18 && s.accetta(voto)){
            d.verbalizza(s, voto);
        }
        terminaEsame(d);
    }
}
```

```
package prova;

import java.util.Random;

public class Docente {

    public int valuta(Studente s) throws InterruptedException{
        Thread.sleep(new Random().nextInt(300));
        return new Random().nextInt(30);
    }
    public void verbalizza(Studente s, int voto){
    }
}
```

```
package prova;

import java.util.Random;

public class Studente {
    public boolean accetta(int voto){
        return new Random().nextBoolean();
    }
    public void partecipaAEsame(Aula a){
        new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    a.sostieniEsame(Studente.this);
                } catch (InterruptedException e) {
                    // TODO Auto-generated catch block
                    e.printStackTrace();
                }
            }
         }).start();;
    }
}
```

Esercizio 4

Si consideri il seguente metodo statico Java:

```
public static int foo(int a, int b) {
```

```
   int n = 3;

   if (a == 0 || b == 0)
      return 1;
   else
      while (a%n != 0)
         if (a > b) a -= b;
   return b;
}
```

e si definisca:

1. il diagramma del flusso di controllo;
2. un insieme minimo di casi di test che coprano tutte le istruzioni;
3. un insieme di casi di test che copra tutti i branch;

Quali (probabili) errori sono evidenziati dall'insieme di test definito al punto precedente?

PUNTO A.
Lasciato al lettore

PUNTO B.
(A=1, B=0), (A=4, B=1)

PUNTO C.
(A=1, B=0), (A=4, B=1), (A=4, B=2)

Probabili errori: infinite loop per i valori A=4, B=2.